



深圳技术大学

SHENZHEN TECHNOLOGY UNIVERSITY

本科毕业论文（设计）

题目： 上机考试信息安全收发 CS 软件体系设计

姓 名： 范佳铭

学 院： 大数据与互联网学院

专 业： 计算机科学与技术

学 号： 202100202085

指导教师： 郭云镛

职 称： 助理教授

提交日期： 2025 年 5 月 16 日

深圳技术大学本科毕业论文（设计）诚信声明

本人郑重声明：所呈交的毕业论文（设计），题目《上机考试信息安全收发 CS 软件体系设计》是本人在指导教师的指导下，独立进行研究工作所取得的成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式注明。除此之外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。本人完全意识到本声明的法律结果。

毕业论文（设计）作者签名：

日期： 年 月 日

目 录

摘要.....	I
Abstract.....	II
1 引言	1
1.1 课题的研究背景与意义.....	1
1.2 国内外发展状况.....	2
1.2.1 技术应用现状	2
1.2.2 基于 C/S 架构与对称加密的图书馆管理系统	3
1.2.3 基于 C/S 模式的远程网络视频监控系统.....	4
1.3 论文组织结构	6
2 关键技术概述	7
2.1 C/S 软件体系.....	7
2.2 Electron 技术简介	7
2.3 Node-Media-Server 流媒体服务器.....	8
2.4 FFmpeg 音视频处理工具	8
2.5 WebSocket 实时通信技术.....	10
2.5.1 技术优势与特点	10
2.6 本章小结.....	11
3 需求分析.....	12
3.1 功能需求分析	12
3.2 服务端需求分析.....	13
3.3 本章小结.....	13

4 系统设计与实现.....	14
4.1 客户端设计与实现	14
4.1.1 主进程与应用管理模块	14
4.1.2 媒体服务与信息采集模块.....	16
4.1.3 API 服务与通信模块.....	17
4.1.4 配置管理模块	18
4.2 服务端设计与实现	18
4.3 线下考试无纸化密码分发	20
5 系统测试.....	22
5.1 测试环境配置	22
5.2 功能测试.....	22
5.2.1 客户端功能测试	23
5.2.2 服务端功能测试	23
5.3 性能测试.....	24
5.3.1 客户端性能评估	24
5.3.2 服务端性能评估	24
5.4 基础安全检查	25
5.5 测试结论与展望.....	26
6 总结与展望	27
参考文献	27
致谢.....	30

上机考试信息安全收发 CS 软件体系设计

【摘 要】 随着在线教育和远程考试的普及，保障考试公平性与安全性成为重要挑战。传统监考方式难以适应线上场景，现有远程监控工具亦存在不足。针对这些问题，本设计旨在开发一个基于 C/S 架构的安全考试信息收发与监控系统。客户端采用 Electron 框架构建，提供图形化配置界面，负责实时捕获考生屏幕与摄像头画面、采集系统信息（如 MAC 地址、IP 地址、CPU 使用率、运行进程等）、并通过 FFmpeg 和 Node-Media-Server 进行视频流处理与本地分发。服务端基于 Golang 和 Gin 框架开发，负责管理客户端连接与身份验证、显示客户端连接状态、分发视频流地址、通过 WebSocket 实现与客户端的实时双向通信（如下发指令、接收状态信息、发送可设置优先级的通知并记录历史）。系统还关注了安全性，采用加密存储本地配置、安全分发服务器访问凭证、Token 认证等机制，并设计了禁止随意退出、单实例运行等防作弊措施。通过在 Windows 和 macOS 环境下进行功能、性能、兼容性、安全性及弱网测试，结果表明该系统功能完备、性能稳定、兼容性良好、安全性高，能够有效满足上机考试或竞赛的实时监控需求，提升监考效率与考试公平性。

【关键词】 在线考试；远程监考；C/S 架构；Electron 框架；流媒体；信息安全

Design of C/S Software Architecture for Secure Information Transmission in Computer-based Examinations

【Abstract】 With the popularization of online education and remote examinations, ensuring fairness and security has become a significant challenge. Traditional proctoring methods struggle to adapt to online scenarios, and existing remote monitoring tools have limitations. To address these issues, this design aims to develop a secure examination information transmission and monitoring system based on the C/S architecture. The client-side, built with the Electron framework, provides a graphical configuration interface, responsible for real-time capture of the examinee's screen and camera feed, collecting system information (such as MAC address, IP address, CPU usage, running processes, etc.), and processing/distributing the video stream locally through FFmpeg and Node-Media-Server. The server-side, developed with Golang and the Gin framework, is responsible for managing client connections and authentication, displaying client connection status, distributing video stream addresses, and implementing real-time bidirectional communication with clients via WebSocket (such as sending commands, receiving status information, sending notifications with configurable priority and recording history). The system also focuses on security, employing mechanisms such as encrypted storage of local configuration, secure distribution of server access credentials, Token authentication, and designing anti-cheating measures like prohibiting arbitrary exit and single-instance operation. Through functional, performance, compatibility, security, and weak network tests conducted in Windows and macOS environments, the results show that the system is functionally complete, performs stably, exhibits good compatibility, and offers high security, effectively meeting the real-time monitoring needs for computer-based exams or competitions and improving proctoring efficiency and exam fairness.

【Key words】 Online Examination; Remote Proctoring; C/S Architecture; Electron; Streaming Media; Information Security

1 引言

1.1 课题的研究背景与意义

21 世纪是充满信息化的时代，面对信息化的冲击，各行各业都在不断调整自己的发展策略，通过充分利用信息化技术来提高自身的竞争力。许多我们习以为常的生活方式也在悄然地发生变化，例如我们十分熟悉的教育考试的形式，也在信息化冲击下，逐渐发生改变，一些课程的考试，由传统的纸质考试逐渐变成数字化的上机考试，而如何与时俱进的改进上机考试背景下的监考流程与策略，就变成了当今的重要课题。

这其中最为关键的，就是上机考试的信息安全问题。首先，对于考生来说，参加考试的人员唯一标识如果在考试前被篡改或者泄露，就有可能出现替考作弊的情况。对于部分需要远程连接服务器实操的上机考试，可能会在考试过程中遭遇到网络攻击，对于此等情况，就有一些基于机器学习的入侵检测模型，用于检测 SSH 协议上的用户名枚举攻击^[1]。除此之外，考试中对于考生周边环境以及考试机环境的监控也同样重要，因此，如何在不影响考试机性能表现的情况下，收集必要的信息，也成了本毕设的一大关注点。

需要补充的是，在本毕设的背景下，上机考试主要指的是具备线下考试场所，采用电脑在机房环境下或者是搭建好局域网的大型考试场所进行考试的形式。这样来看，传统的人工监考，在这种上机考试下难以全面的监控考生的电脑数据，而目前现有的远程监控工具往往是配置复杂的。那么基于对上述背景考量，本毕设设计了一个名为上机考试信息安全收发 CS 软件体系设计的系统，这个系统对于上机考试的流程规范化是有着正向的影响。

首先对于参加上机考试的考生来讲，都应被赋予一个唯一的身份标识，通常来讲，这种标识由一个唯一的账号以及密码组成，这个标识需要尽可能的降低泄露风险，其次对于监考方来说，服务器的访问地址以及密码也应被妥善保护。这样的考量，可以提高考试的可信性与公平性。此外，上机考试过程中，电子设备环境的复杂性往往是设计监考方案的一大挑战，如何在不影响考试机性能表现的情况下，能适时地全面的采集考试机进程数据，以及考试机的实时屏幕数据，这方面的合理设计，也可以提高监考的质量以及保证考试公平。

1.2 国内外发展状况

1.2.1 技术应用现状

为了更好地满足当今对于上机考试的需求，国内外也出现了很多的上机考试平台。这些系统主要通过提供诸如登录、发卷、判题、批卷、作弊检测等的功能，来保证上机考试的正常运行。在国外，对于数据相关部分同样也有所研究，例如可以适时地结合并行化 Rsync 工具，来进行数据相关操作^{[2][3]}。在国内，考试信息化系统的发展也已初具规模，以部分高校为例，他们自有的基于 B/S 架构的校内考试平台实现了考试管理、实时成绩反馈等功能。然而，这些系统也存在一些不足：

部分高校的系统在对于考试密码分发的管理上略显欠缺，方便起见，这些系统通常采用的是考前打印密码条分发给考生或者是采用常见的账号密码登录，这些相关的人员标识数据容易发生人为泄露，密码的存储也存在着一定的风险。这部分可以参考基于 MD5 和 SHA-256 的多重哈希方法，用于提升数据完整性验证的安全性，也特别适用于在线考试这种多文件传输和存储的场景^[4]。

目前国内流行的上机考试系统主要是牛客在线考试系统以及部分竞赛采用的 OMS 监考系统。像牛客这种线上考试平台在提供线上笔试、面试等考试功能的同时也提供了一些考试数据的统计分析功能，但是由于面向的考试群体通常比较庞大，所以考试过程中的指令下发、屏幕监控以及考试机进程数据的收集分析等功能的实时性表现不佳，具体表现为考中对于违规行为不能实时反馈警告，没有足够权限获取考试机的进程数据以及屏幕监控质量较低。这些平台对于早期常见的如网络拥塞、时间控制、身份验证等问题，都有相应的解决策略^[5]。相比之下，竞赛常用的 OMS 监考系统在实时监控方面表现较好，它采用了远程双机位监考的方式，还额外支持进程检测与拦截，不过仍存在可订制项少、登录流程较复杂、支持类型多为线上考试，对线下支持不完善以及跨平台兼容较差的问题。

国内在其他领域采用 C/S 架构的方案有很多，例如有一款面向电力设备在线监测的应用程序，这使得管理和分析设备更为简单，一款铁路货运专线管理的一体化信息系统也采用了这种架构^{[6][7]}。国外考试信息化起步较早，一项研究指出一种使用电子标识符进行远程计算机用户认证的方法，基于微软标准实现的远程桌面协议 (RDP)，并通过虚拟通道保护通信信道的安全性^[8]，同时，也提出了利

用机器学习从 Windows RDP 事件日志中检测横向移动攻击的方法，并展示了所提模型在分类 RDP 会话方面的优越性能^[9]。

国外在上机考试领域有着不少的成熟案例，例如，在被广泛采用的在线考试平台中，以 ProctorU 和 Examity 为代表的在线考试平台在安全性和实时性方面具有领先优势，ProctorU 通过实时屏幕共享和 AI 驱动的行为分析技术，可实时监控考生的屏幕操作和周围环境，及时检测作弊行为。Examity 则有着采用多层次身份验证（如生物识别和行为模式分析）和加密数据传输技术的方案，来确保考生信息的安全。

截至目前，关于如何保证线上考试的公平，通常可以用多层身份验证、题目与选项随机化、远程多机位监考和算法自动检测来解决。多层身份验证作为最基础的防作弊措施，一定程度上能确保参与考试的是本人，例如，部分学校通常提供账号密码登录系统，并安排人工核验认证比对，有时，在考试进行中系统还会随机抽检进行人脸识别，防止中途更换考生。在一些重要的竞赛中，还会采用多机位监考，这是目前高规格考试常使用的措施，通常包含三路音视频监控，即 PC 录屏、电脑摄像头以及手机副摄像头，实现 360 度全方位监考。监考人员可以通过实时音视频监控全面观察考生的行为，包括面部表情、手部动作及周围环境，从而有效预防和检测作弊行为。算法自动检测则是更进一步的手段，它利用算法和人工智能技术分析学生的答题行为，及时发现可疑的作弊行为。例如，监考工具可以检测学生是否打开了非考试相关的网页或应用程序，同时记录如多个人脸、无人脸、切屏等异常情况。一旦发现作弊行为，系统会自动记录并提醒监考人员进一步调查。

此外，还不乏有诸如结合 RSA 加密与压缩隐写技术实现的一种高效的数据隐藏方案，通过构建一个预测 SSH 流量行为的模型，实现了对 SSH 通信模式的有效识别^[10]以及通过引入包间到达时间特征及生成对抗网络（WGAN-GP）算法来减少误报率等研究，共同促进提升上机考试的安全性^{[11][12]}。

1.2.2 基于 C/S 架构与对称加密的图书馆管理系统

目前，图书馆中对于一些图书下架的操作，依赖于上级教育部门或者是国家机密文件下发，传统的基于高校组织人员人工核查速度慢，请供应商协助可以提高效率，但是可能存在泄密的风险^{[13][14][15]}。

这种图书馆管理系统采用了基于 C/S 的架构，实现了对于部分不良图书的自动核查以及下架功能，同时，对于图书馆内部文件实现了动态加密，保密性强。对于内部的机密文件，采用了对称加密技术，提高了数据的安全性^[16]。这种设计思路下，对于每个高校的图书馆，都视作为一个客户端，图书馆中每本书的信息经由管理员录入软件，此时软件本地核查服务端下发的不良图书名单，自动筛选需要下架的图书，生成一份下架名单。服务端则由各自图书馆的上级部门持有，服务端需要维护图书馆馆藏数据，负责各类图书的定性工作。这种技术在成都师范学院图书馆有所应用，支持筛选有害图书，并建立了基于对称加密技术的 Oracle 服务端，实际落地了 C/S 系统架构的应用。

此处这种基于 C/S 架构的图书馆管理系统，相比传统人工核查更具备自动化、高效、安全等的优势，尤其是图书馆馆藏很大的情况下，数据量会很大，这种架构可以利用本地客户端的性能来高效地处理这些馆藏数据。

1.2.3 基于 C/S 模式的远程网络视频监控系统

传统的数字化模式的视频监控系统在安全性要求较高的场景略显乏力，这种基于 C/S 模式的网络监控系统会将远程采集的数据，通过本地设备高性能运算加密算法，再使用特殊的传输通道，将加密后的数据传输到接收端，解密后再将数据传输到显示终端上^[17]。在传输过程中，传统的视频编码不能很好的将数据压缩至合适的码率，导致在终端处显示的画面十分模糊，这里采用了基于 H.264 的编码技术，使得图像质量以及压缩率有了更好地表现。

H.264 是 MPEG-4 标准所定义的最新格式，是 MPEG-4 标准的第十部分，有时也称为 AVC^[18]，它的编解码流程主要包含有变换（Transform）和反变换、量化（Quantization）和反量化、环路滤波（Loop Filter）、熵编码（Entropy Coding）^[19]。此外，该编码技术还采用了许多新技术，如帧内预测^[20]，帧间预测^[21]，多参考帧运动估计^[22]，1/4 像素和 1/8 像素精度预测^[23]，4×4 整型变换等技术^[24]，使得编码效率加快，在保证图像质量的情况下增加了视频压缩码率。

那么案例中所采用的 H.264 编码技术，也是在本毕设采用的编码技术，虽然目前已存在更为先进的 H.265 技术，但是在本毕设的上机考试局域网环境下，H.265 所带来的高压缩率提升并不明显，反而会带来一系列的兼容问题，本毕设主要工作就是为了提高兼容性，同时 H.265 对于专利的管理更为严格，在部署时可能会

出现一系列不便，就比如目前使用的 Windows11 平台，如果要使用 H.265 编解码，需要安装额外付费的编解码软件，因此这里采用了 H.264 作为本毕设的编码技术选择。

1.3 论文组织结构

第一章为引言部分，主要介绍了选题的背景与意义，分析了国内外上机考试系统发展状况，也指出了本毕设的技术选择原因。

第二章为关键技术概述，主要介绍了本系统采用软件架构，以及开发软件采用的技术框架。

第三章为设计需求分析，对于上机考试流程进行了分析，从而确定了本毕设的操作流程逻辑。

第四章为系统设计与实现，主要展示了本系统客户端交互界面以及操作逻辑，以及服务端的设计思路，最后展示了服务端的控制面板。

第五章为系统测试，首先介绍了本系统的开发环境以及 C（客户端）、S（服务端）两端的测试环境，接着利用 ApiFox 对服务端进行了性能测试，结果表明服务端的接口响应时间符合上机考试的实时性要求。

第六章总结与展望，总结了本设计的具体工作，也指出来本系统的优势以及在部分方面的不足，同时也对未来上机考试系统的发展进行了展望。

2 关键技术概述

2.1 C/S 软件体系

C/S（客户端/服务器）软件体系是一种经典的软件架构模式，它将软件分为客户端和服务端两部分，通过职责分离实现系统功能。客户端主要负责用户界面呈现、交互逻辑处理和部分数据验证，而服务端则集中处理核心业务逻辑、数据存储管理和安全控制。这种架构模式自 20 世纪 80 年代末期开始流行，经过多年发展已成为企业级应用的主流架构之一。这种架构因其高效的数据传输机制和强大的交互能力，在许多领域得到了广泛应用，如企业资源规划（ERP）系统、客户关系管理（CRM）系统、在线游戏、数据库管理工具等。随着技术的发展，C/S 架构也在不断地与云计算、大数据、微服务等新技术融合，以适应新的应用场景和需求。

此外，随着技术的演进，传统 C/S 架构也在不断发展，出现了富客户端（Rich Client）、胖客户端（Fat Client）和瘦客户端（Thin Client）等变体形式，以及混合型架构如三层架构（Three-tier Architecture）和分布式架构（Distributed Architecture）等，进一步提升了系统的灵活性和适应性。在当代软件开发中，C/S 架构虽面临 Web 应用的挑战，但在特定领域如高性能计算、复杂业务处理、实时交互等场景中仍具有不可替代的优势。

2.2 Electron 技术简介

Electron 是由 GitHub 开发的一个开源框架，允许开发者使用 Web 技术来构建跨平台的桌面应用程序。其核心思想是将 Chromium 和 Node.js 集成在一起，使得开发者能够在桌面端实现与 Web 一致的界面和交互体验。这样的架构使得 Electron 应用可以在 Windows、macOS 和 Linux 等多个平台上运行，具有很高的跨平台兼容性，Node.js 的引入，使得 Electron 软件同时也具备强大的本地功能和性能，不仅弥补了 Web 应用在桌面端的不足，也使得客户端的跨平台开发变得更加容易。

本系统采用 Electron 作为客户端的开发框架，它可以承担考试监控工具的界面渲染、数据采集、加密存储等关键任务。此外，通过对 Node.js 的调用，可以实现对本地屏幕、摄像头等硬件资源的访问。这些优势，使得它能很好的承载本系统的需求。

2.3 Node-Media-Server 流媒体服务器

NodeMediaServer 是基于 Node.js 实现的轻量级流媒体服务器，它支持 RTMP、HTTP-FLV、WebRTC 等主流流媒体协议，它采用了模块化的设计，具有高度的可扩展性、定制化能力和庞大的社区生态，同时在当前开源流媒体服务器的解决方案中性能表现也是较为出色的。

在本系统中，通过 RTMP 协议实现毫秒级的视频流传输，可以保证监考画面的实时性。在实际测试中，端到端延迟也可控制在毫秒级的范围内，满足实时监控的严格要求。同时，支持 HTTP-FLV 协议，可在网络条件不佳时提供更稳定的传输能力。同样的，它也是基于 Node.js 开发，可在 Windows、Linux、macOS 等多个平台上运行，与 Electron 应用完美配合。这种跨平台特性使得监考系统可以在不同操作系统环境下保持一致的性能表现，大大降低了部署和维护成本。此外，它还采用了事件驱动的非阻塞 I/O 架构，即使在处理多路视频流时也能保持较低的系统资源占用。在实际应用中，单核处理器即可支持数十路 720p 视频流的并发处理，非常适合在考生端本地部署使用，不会明显影响考试过程中的系统性能。

因此在本系统中，采用 Node-Media-Server 作为客户端的本地流媒体服务器，负责接收和处理来自 FFmpeg 的屏幕和摄像头视频流数据，并提供稳定的流媒体分发服务。在必要时，还可以对视频流进行必要的处理（如添加水印、调整码率等）后再分发给监考服务器。其高效的流处理能力和稳定的性能表现，为实时监控提供了可靠的技术支持。

2.4 FFmpeg 音视频处理工具

FFmpeg 是一个功能强大的开源音视频处理工具集，由 FFmpeg 项目组于 2000 年开始开发，目前已成为业界最为广泛使用的多媒体处理框架。它提供了完整的音视频编解码、转码、混流、滤镜和推流等功能，支持几乎所有常见的音视频格式，同时，它也是跨平台的，具备良好的兼容性。在本系统中，FFmpeg 主要用于屏幕录制和摄像头画面的采集与推流，是实现实时监控功能的核心组件之一。

FFmpeg 支持同时采集屏幕画面和摄像头视频，并通过高级滤镜链进行画中画合成处理。在监考场景中，可以将摄像头画面作为小窗口叠加在屏幕画面上，实现对考生行为和屏幕内容的同步监控。具体实现上，可以使用 FFmpeg 的 overlay

滤镜完成画面合成，示例命令如 2.1 所示。正如引言中提到的，本系统考虑到提高兼容性，所以采用了 H.264 编码，并针对实时性场景进行了指令优化。在实际情况中视频质量和传输延迟需要进行平衡，因此，在开发中，对 FFmpeg 运行参数做了如图 2.2 调整。

```
# 使用FFmpeg捕获桌面和摄像头视频，合成后推流到RTMP服务器
ffmpeg \
    # 捕获桌面画面
    -f gdigrab \                # 使用gdigrab采集器(Windows系统专用)
    -framerate 15 \            # 设置帧率为15fps
    -i desktop \                # 输入源为桌面
    # 捕获摄像头画面
    -f dshow \                  # 使用DirectShow采集器
    -i video="Webcam" \         # 输入源为名为"Webcam"的摄像头设备

    # 视频处理滤镜配置
    -filter_complex "\
        [1:v]scale=320:-1[cam];    # 将摄像头画面缩放到宽度320像素，高度自适应
        [0:v][cam]overlay=main_w-320:0 # 将摄像头画面叠加到桌面右上角
    " \

    # 编码和推流设置
    -c:v libx264 \              # 使用H.264编码器
    -preset ultrafast \         # 使用最快的编码预设，降低延迟
    -tune zerolatency \         # 优化设置以降低延迟
    -f flv \                    # 输出格式为FLV
    rtmp://localhost/live/stream # 推流地址可任意更改
```

图 2.1 FFmpeg 叠加命令示例

```
// Windows 系统下捕获桌面屏幕并推流到 RTMP 服务器的 FFmpeg 命令
[
    // 输入设置 - 使用 gdigrab 捕获 Windows 桌面
    "-f",
    "gdigrab",                // 指定输入格式为 gdigrab (Windows 专用的桌面捕获器)
    "-framerate",
    "30",                      // 设置捕获帧率为 30fps
    "-i",
    "desktop",                // 指定输入源为整个桌面

    // 视频编码设置
    "-c:v",
    "libx264",                // 使用 H.264 编码器
    "-preset",
    "ultrafast",              // 使用最快的编码预设，降低 CPU 占用，减少延迟
    "-tune",
    "zerolatency",            // 优化设置以最小化延迟，适合实时流

    // 质量和日志设置
    "-q:v",
    "28",                     // 设置较低的视频质量参数，降低质量以减少网络压力
    "-loglevel",
    "warning",                // 只显示警告和错误信息，不输出帧质量等详细日志

    // 输出设置
    "-f",
    "flv",                    // 设置输出格式为 FLV，适合 RTMP 流媒体传输
]
```

图 2.2 FFmpeg 叠加命令示例

FFmpeg 支持 NVIDIA NVENC、Intel QuickSync、AMD AMF 等硬件编码技术，

为了降低考试机的性能要求，本系统启用了硬件加速，可以将视频编码的 CPU 占用从 30%-40% 降低到 5%-10%，有效减少对考试机器性能的影响。在监考系统的实际部署中，FFmpeg 作为独立进程运行，由 Electron 应用通过进程间通信进行控制和管理。这种架构设计既保证了视频处理的高效性，又避免了因视频处理导致的主应用卡顿，为监考系统提供了稳定可靠的音视频处理基础。

2.5 WebSocket 实时通信技术

WebSocket 是一种在单个 TCP 连接上进行全双工通信的协议，由 IETF 在 RFC6455 中标准化，设计用于解决传统 HTTP 请求-响应模式在实时通信场景中的局限性。WebSocket 协议在 2011 年成为标准，目前已被广泛应用于即时通讯、在线游戏、金融交易等需要低延迟双向通信的场景。在本监考系统中，WebSocket 作为核心通信技术，用于实现客户端与服务器之间高实时性数据交换的需求。

2.5.1 技术优势与特点

WebSocket 技术具备持久连接与低延迟、双向实时通信、资源占用低、跨平台兼容性、安全性保障、可扩展性强等优势。双方建立 WebSocket 连接后可持续双向实时通信，避免频繁的连接建立和断开，消息传递延迟通常在 50ms 以内，远低于传统 HTTP 轮询的 300-500ms，同时，服务器可以主动向客户端推送消息，无需等待客户端轮询请求，这使得监考人员能够实时下发指令（如要求考生调整摄像头、停止可疑行为等），同时考生端也能立即报告异常状况（如网络波动、设备故障等）。在数据格式上，WebSocket 支持文本和二进制两种基本数据类型，并可通过应用层协议（如 JSON、Protocol Buffers 等）实现更复杂且占用更低的数据结构传输，相比于 HTTP 请求，WebSocket 帧头部更加精简，通常只有 2-14 字节，这大幅减少了网络带宽和服务器资源的占用。实际测试表明，在 1000 名考生同时在线的场景下，WebSocket 方案的服务器 CPU 占用率比 HTTP 轮询低约 60%，网络流量减少约 70%。这一优势使得系统能够支持更大规模的考试监控，同时降低了基础设施成本。更为重要的是，所有现代浏览器和主流开发框架（包括 Electron）均原生支持 WebSocket 协议，确保了系统在不同平台和环境下的一致性表现。本毕设系统的客户端利用了 Electron 内置的 WebSocket API 进行通信，实现了无缝集成。

2.6 本章小结

本章节主要介绍了本毕设系统开发时采用的 C/S 软件架构、跨平台的 Electron 框架、提供本地流媒体服务的 Node-Media-Server 流媒体服务器、音视频处理工具 FFmpeg 和提供考试指令交互的实时通信技术 WebSocket。通过这些技术，本系统才能够实现高效、稳定、可靠的监考功能，并为上机考试提供实时监控的服务。

3 需求分析

在介绍本系统各个功能之前，需要先拆分面向用户的需求和面向系统后台处理的需求。首先需要明确的是，本系统的主要用户是考生、志愿者以及监考人员，对于考生而言，需要关注的点应该是考试本身，与本系统相关的主要是监考过程中的指令下发。对于志愿者而言，他可能需要配置好机器。对于监考人员，则是每个考生的屏幕内容以及指令下发。

在本毕设的上机考试背景下，应有志愿者这一角色进行考试机器的配置。对于志愿者而言，需要配置服务器的通信地址、访问密码、考试机器号以及考生信息。这里的服务器的通信地址、访问密码是为了提高本系统的泛用性，方便不同的考试场景使用。

为了尽最大限度的降低本系统对考生参加考试的影响，考生在考试全程需要用到本系统的功能应只有接收并回应监考人员的指令，因此，这里采用了系统弹窗来实现指令通知，同时，在志愿者配置完机器后，本系统将会最小化，来减少系统监控对于考生考试的干扰。

3.1 功能需求分析

为了提供直观的配置页面，客户端部分提供一个图形化的配置页面，方便志愿者进行考前机器配置。在成功连接上服务器之后，图形化页面自动最小化进入后台运行。当本系统进入到后台，也就是考生信息录入之后，后台将自动建立一个由 Node-Media-Server 提供服务的本地流媒体服务器，并通过 FFmpeg 实时捕获考试机屏幕上的所有内容以及摄像头的内容（如果有），同时建立一个 WebSocket 连接，方便监考指令的下发，此外，考试机的流地址交由本地流媒体服务器分发给服务器。

考虑到电子设备需要一些诸如：MAC 地址、IP 地址、CPU 使用率、后台运行程序等信息来辅助监考员判断考试机状态，所以，本系统还需要一个系统信息采集服务，并定期发送系统信息到服务器，这部分内容可以根据服务器的需求决定是否采集额外的信息。为了保证监考系统的正常运转，本系统应不允许考生随意终止程序，需要输入（由服务端验证的）特定密码才能退出，此外，系统还需要防止多实例运行，确保监控的唯一性和持续性。

3.2 服务端需求分析

服务端作为整个监控系统的管理中心，负责汇聚和处理来自各客户端的数据流，并提供统一的监控界面。它需要支持管理客户端连接、视频流接收与分发、消息通知管理等功能。具体来讲，服务端需要展示各个考试客户端的连接状态，并对客户端的接入请求进行身份验证和授权管理，同时应能实时显示所有在线客户端的基本信息，便于监考人员掌握考场整体状况。接着，还需要接收并汇总来自客户端的流地址，提供选择界面让监考人员选择查看特定考生的监控画面，对于一些对外开放的大型竞赛，也支持将选中的视频流转发至第三方直播工具（如 OBS）。最后，服务端应提供向指定单个或全体考生发送通知的功能，支持设置消息优先级，并记录消息历史，便于后续审查和分析。

3.3 本章小结

本章节主要分析了本系统在开发时需要满足的用户需求和系统需求，主要分为面向用户的需求和面向系统后台处理的需求。对于用户而言，需要有直观地图形化页面来交互，在后台数据处理则是需要考虑到各种情景。

4 系统设计与实现

本系统采用 C/S 架构设计，分为客户端和服务端两个主要部分。客户端基于 Electron 框架开发，利用 Node.js 的底层能力和 Chromium 的界面渲染能力，负责考试机的屏幕捕获、视频推流和系统监控；服务端基于 Golang 开发，采用 Gin 框架实现 Web 服务，负责客户端管理、视频流分发和监考指令下发，同时采用前端技术实现了简易的监考管理界面。

4.1 客户端设计与实现

客户端采用模块化设计，基于 Electron 构建，整合了 Node.js 的后端能力与 Chromium 的前端界面展示能力。其核心逻辑主要在 Electron 的主进程中实现应用的生命周期管理、系统交互、调用 Node.js 的底层能力，在渲染进程中实现与用户交互的界面展示，避免因数据处理导致的界面卡顿。

4.1.1 主进程与应用管理模块

此模块是客户端应用的入口点和核心控制器，基于 Electron 的 `app` 和 `BrowserWindow` 模块构建，它主要实现了应用生命周期管理，通过监听 `app` 对象的生命周期事件来管理应用状态。例如，在 `app.on('ready', callback)` 事件触发后，完成应用的初始化，包括创建主窗口 (`BrowserWindow`)、加载渲染进程的 HTML 文件以及创建系统托盘图标。通过 `app.requestSingleInstanceLock()` 方法确保系统只运行一个客户端实例，防止多开影响监控。为确保应用在所有窗口关闭后仍能在后台运行（特别是进行推流和监控），此模块还监听了 `app.on('window-all-closed', callback)` 事件，也就是退出软件的事件，它阻止了默认的退出行为，除非用户通过托盘菜单输入退出密码显式退出。此模块的生命周期如图 4.1 所示。

渲染进程创建的主窗口承载了用户交互界面（如登录、状态显示）。为了防止考生意外或故意关闭监控，主窗口的关闭行为被拦截：监听窗口的 `close` 事件，并通过 `event.preventDefault()` 阻止窗口关闭，转而调用 `mainWindow.hide()` 将窗口隐藏，同时依赖系统托盘 (`Tray`) 进行后续交互。系统托盘图标使用 `Tray` 模块创建，并关联了一个上下文菜单 (`Menu.buildFromTemplate`)，提供“显示窗口”和需要密码验证的“退出程序”选项。图形化的考前配置界面如图 4.2 所示。

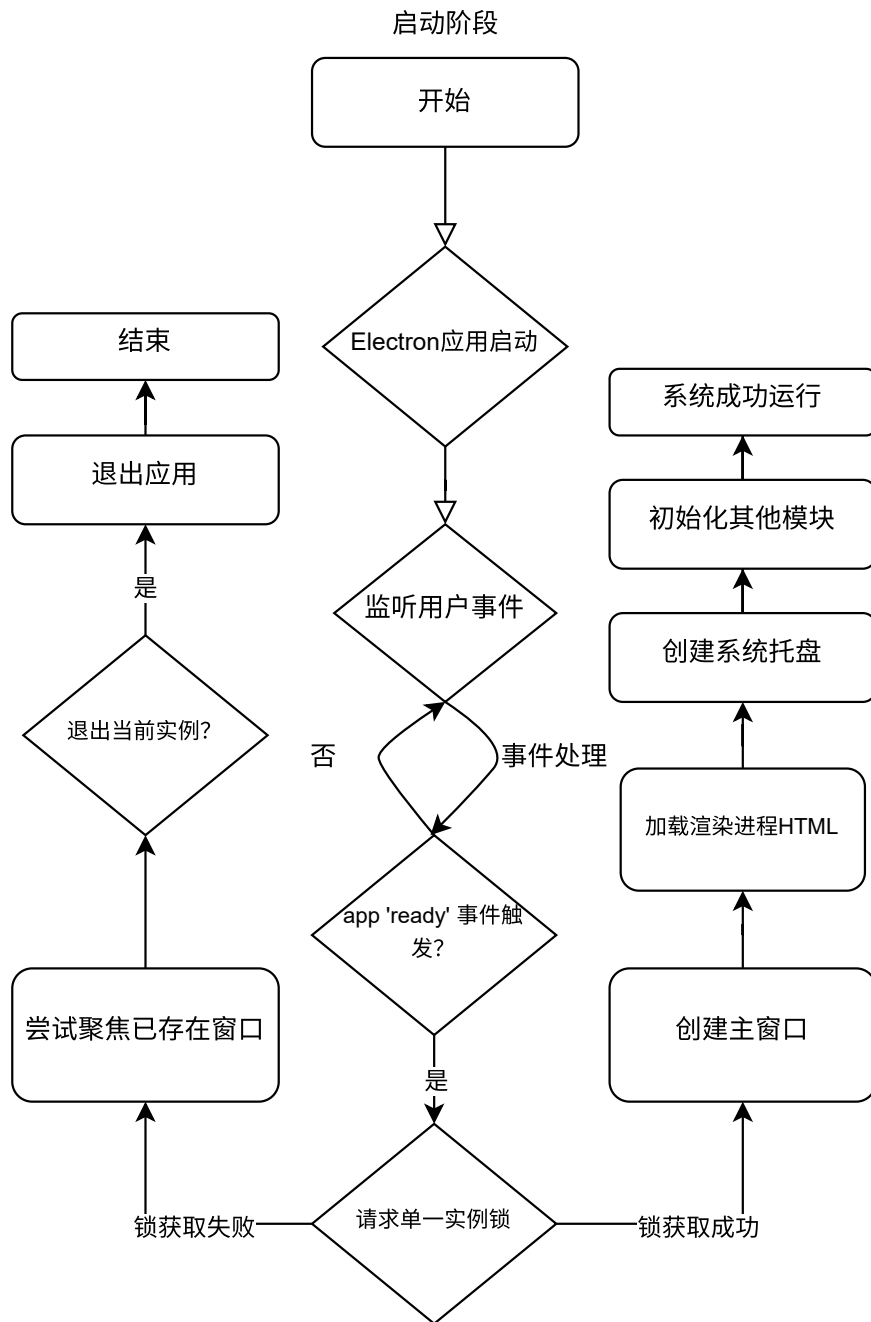


图 4.1 客户端应用生命周期管理流程图示例

主进程与渲染进程（负责 UI 展示）之间通过 Electron 的进程间通信机制 (ipc-
Main 和 ipcRenderer) 进行交互，这样可以缓解因数据处理事件阻塞时导致的软件”
卡死”现象。例如，渲染进程通过 ipcRenderer.send(channel, ...args) 发送登录信息

服务器IP地址

例如: 192.168.1.100

服务器端口

例如: 8080

访问密码

请输入服务器访问密码

考试机号

例如: 14

登录

图 4.2 考前配置图形化界面

或配置信息，主进程通过 `ipcMain.handle(channel, listener)` 或 `ipcMain.on(channel, listener)` 接收并处理这些请求，如验证配置、启动服务等。反之，主进程也可以通过 `mainWindow.webContents.send(channel, ...args)` 向渲染进程发送状态更新或服务器指令。

4.1.2 媒体服务与信息采集模块

该模块是实现考试监控的核心，负责屏幕和摄像头画面的采集、编码与推流、本地流媒体服务以及系统信息采集。本系统通过调用跨平台的 FFmpeg 技术，并结合 Node.js 后端能力来创建一个独立的 FFmpeg 子进程来执行采集和推流任务，避免了复杂的 C++ 绑定和潜在的性能瓶颈。此外，还在在不同的操作系统平台上，采用动态构建 FFmpeg 启动命令的方式，来适配不同平台不同的系统采集能力。在调用 FFmpeg 时，本系统会尝试采用硬件编码来降低延迟和系统占用，生成的本地流指向 Node 的媒体服务器。推流使用的 `streamKey` 则是与考生的唯一标识关联。客户端主进程负责管理 FFmpeg 子进程的生命周期，监听其各类事件。在开发过程中可以解析 FFmpeg 输出获取推流状态和错误信息，当进程发生意外

退出时会尝试自动重启 FFmpeg 进程以保证监控的连续性。以上生成的数据流依赖于一个在客户端本地运行的 Node-Media-Server (NMS) 实例。FFmpeg 将处理后的视频流推送到本地 NMS，NMS 负责接收这个本地 RTMP 流以及其他可用的如 HTTP-FLV 或其他协议的转播地址，这样做的好处是将流处理的复杂性与客户端核心逻辑解耦，并且 NMS 相对稳定。客户端随后会将这个本地流的播放地址（如：<http://127.0.0.1:8000/live/streamKey.flv>）报告给服务端。

图 4.3 是本系统开发时使用的预览界面，其中可以看到 FFmpeg 叠加屏幕内容以及摄像头内容后最终画面，预览界面下是生成的屏幕流地址以及摄像头流地址，图片右侧的是当前设备可以使用的屏幕捕获设备、摄像头捕获设备以及当前系统正在运行的进程。



图 4.3 本地捕获能力预览界面

4.1.3 API 服务与通信模块

此模块负责客户端与考试服务器之间的通信请求，其中，对于实时性要求较高的监考指令下发与考试机系统信息部分，采用了 WebSocket 协议。使用 ws 库创建 WebSocket 客户端实例。根据配置管理模块提供的服务器地址发起的连接，来监听 open, message, error, close 等事件来管理连接状态。open 事件表示连接成功建立，此时客户端会发送身份验证信息（如携带 token 或考试机编号）。message 事件的回调函数负责接收来自服务器的消息。这部分的消息采用 JSON 格式，客户端收到后使用 JSON.parse() 解析。根据消息中的 type 字段或类似标识，执行不同操作，例

如：处理服务器下发的监考指令（如 `lockScreen`, `showWarning`, `requestSystemInfo`）、响应心跳包、更新本地状态等。客户端也主动通过 `WebSocket` 发送消息，例如：定期发送心跳包、上报系统基本信息、报告本地推流地址、响应服务器指令的结果等。

当连接出现关闭或者是意外终止时，`WebSocket` 的 `close` 或 `error` 事件将会触发，这里客户端实现了自动重连逻辑：使用 `setTimeout` 设置一个延迟，当延迟时间结束后仍未检测到连接活性时，会重新尝试建立 `WebSocket` 连接。这种机制确保了在网络暂时不稳定的情况下，客户端能够自动恢复与考试服务器的通信。

4.1.4 配置管理模块

该模块负责存储和管理客户端运行所需的配置信息，如服务器地址、认证凭据、考试机标识等。本系统使用了 `electron-store` 库进行本地配置的持久化存储。该库提供了一个简单的键值对 API (`store.set(key, value)`, `store.get(key)`)，并将数据存储在与用户操作系统的标准应用配置目录下。为了增强安全性，存储在本地敏感配置信息（特别是服务器访问凭证）进行了加密处理。本系统采用 Node.js 内置的 `crypto` 模块实现 AES-256-CBC 对称加密。在存储配置前，使用 `crypto.createCipheriv(algorithm, key, iv)` 创建加密器对敏感值进行加密；在读取配置后，使用 `crypto.createDecipheriv(algorithm, key, iv)` 进行解密。

当系统启动时，首先会尝试从 `electron-store` 加载配置。如果配置不存在或不完整，将会引导用户进入配置界面（通过渲染进程实现）进行输入。加载配置后，会对关键信息（如服务器地址格式）进行基本验证。配置信息（特别是认证凭据）在建立 `WebSocket` 连接时发送给服务器进行最终验证。

4.2 服务端设计与实现

服务端作为整个监控系统的中心枢纽，负责管理所有接入的客户端、处理实时数据流、响应监考指令并提供管理界面。考虑到系统对实时性、稳定性和后续扩展性的要求，服务端采用 Go 语言进行开发，并选用 `Gin Web` 框架构建基础服务。整体架构倾向于模块化设计，便于功能迭代与维护。

首先是 `Web` 服务模块，它基于 `Gin Web` 框架构建，是服务端与外部交互的主要入口，负责处理 `HTTP` 请求。它不仅为监考人员提供如图 4.4 所示的管理界面，

还提供了一系列 RESTful API 接口，用于客户端认证、信息查询、状态管理等操作。Gin 框架以其高性能和简洁的 API 设计，能够有效处理高并发请求，并提供了中间件支持，便于实现日志记录、跨域处理 (CORS)、身份验证等通用功能。关键 API 接口功能示例如下表 4.1 所示。



图 4.4 监考控制面板

表 4.1 服务端核心 API 接口示例

路径	方法	主要功能
/api/auth/login	POST	客户端或管理员登录认证
/api/clients	GET	获取在线客户端列表及状态
/api/clients/{id}/stream	GET	获取指定客户端的流媒体播放地址
/api/clients/{id}/info	GET	查询指定客户端的详细系统信息
/api/command	POST	向指定或全部客户端下发指令
/api/config	GET/POST	管理系统全局配置

WebSocket 服务模块是为了满足监考过程中的实时通信需求（如指令下发、状态上报、心跳维持），服务端实现了 WebSocket 服务。此模块负责管理所有客户端

的 WebSocket 长连接，处理双向数据帧。主要功能包括：维护连接池，跟踪客户端在线状态；高效地向单个、部分或所有客户端广播或定向推送消息（如监考指令、通知）；处理客户端上报的实时系统信息和异常事件；通过心跳机制检测并清理无效或断开的连接，保证连接的稳定性。

客户端管理模块负责维护客户端的整个生命周期信息，它记录了每个已连接客户端的标识、状态（在线、离线、异常）、登录时间、IP 地址、上报的系统信息等。当客户端通过 WebSocket 连接或断开时，该模块会更新其状态。流媒体管理模块与客户端的媒体服务模块对应，核心职责是管理所有客户端上报的流媒体地址。它接收客户端报告的本地流地址（如 HTTP-FLV 地址），并将其与客户端标识关联存储。监考人员通过 Web 界面请求查看某个考生的监控画面时，Web 服务模块会调用此模块获取对应的流地址，并返回给前端播放器。这些模块协同工作，共同构成了服务端的完整功能。通过合理的模块划分和技术选型，旨在构建一个稳定、高效且易于管理的远程监考系统后端。

4.3 线下考试无纸化密码分发

除了通用的功能需求外，针对特定的考试场景，认证和配置流程需要特别考虑。在本毕设上机考试的背景下（如 C 语言期末上机考试、XCPC 程序设计竞赛），如何在保障安全性的前提下，高效地完成大量考试机的登录认证是一个挑战。截至目前，市面上针对此类上机考试场景，存在多种密码或凭证分发方式：

首先是纸质密码条分发，这种方案会将包含登录账号密码的纸条提前放置在每个座位上，或在考试开始时统一分发。优点是简单直接，但缺点是准备工作繁琐，纸条易丢失或被他人窥视，安全性较低。当参考人数过多时，大量的密码条会分散在各个区域，造成管理困难。此外，也存在监考员在考试开始时通过口头或白板告知统一的登录密码。这种方式仅适用于密码一致且安全性要求不高的场景，且容易造成现场混乱和信息泄露。部分高校可以凭借自有的人员管理系统接口生成预设账号，管理员提前为每台机器配置好特定的考试账号和密码，也可直接登录。这种方式对管理员工作量要求大，且如果密码简单或固定，可能会存在安全隐患。

通过分析这些方案的优缺点以及上机考试通常会有志愿者或工作人员提前布置考场的实际情况，本系统在设计客户端时考虑了一种结合人工配置的优化

流程，即将电脑配置以及系统登录的流程融入到志愿者的操作中。具体来看就是在考前，负责布置考场的志愿者可以使用本系统的客户端，通过私密群聊（如微信）与监考人员进行交流，监考人员告知各个志愿者负责的机位以及需要配置的信息，提前为考试机器登录系统。这样，当考生到达指定座位时，考试监控客户端已经处于登录并就绪的状态，考生只需关注考试本身即可，无需进行额外的登录操作。

这种方案的优势在于可以简化考生操作，考生无需记忆和输入密码，降低了考前准备的复杂度，减少了可能因考前密码混淆导致的意外耗时。并且相较于直接将密码暴露给考生，由受信任的志愿者操作，可以在一定程度上减少密码在开考前泄露的风险。这种方案利用现有流程，将认证步骤融入到志愿者本就需要进行的考前机器配置流程中，没有增加额外的角色或复杂的系统。

当然，该方案也存在局限性，它强依赖于线下有足够的人力（志愿者或工作人员）来完成配置和预登录，对于完全远程的线上上机考试场景，可能仍依赖市面上已有的成熟系统。因此，对于线上上机考试，仍需考虑传统认证方式，例如通过安全的邮件系统向考生预发送一次性密码或登录链接，或者利用手机扫描二维码进行身份验证和登录，高校也可集成现有的单点登录（SSO）系统等。

目前已有的方案并不都能满足所有场景，随着技术的发展，在可见的未来可以结合生物识别技术（如人脸识别登录）或更安全的无密码认证方案（如 FIDO/WebAuthn），这可能是进一步提升考试登录环节安全性和便捷性的发展方向。

5 系统测试

系统测试阶段旨在验证所开发的远程监考系统是否满足设计需求，具备预期的功能、性能、稳定性和安全性。考虑到项目的实际进度和资源，本阶段的测试工作主要集中在 Windows 操作系统环境。测试方法结合了功能性测试、性能评估和基础安全检查。

5.1 测试环境配置

为模拟实际部署场景，测试在不同硬件配置的 Windows 11 系统上进行。本测试使用的主要硬件环境配置如表 5.1 所示，服务端的硬件配置如表 5.2。网络环境则模拟了常见的有线和无线连接，并在可能的情况下引入了延迟和丢包，以评估系统在不同网络条件下的表现。

表 5.1 主要测试环境配置

测试项	测试环境
操作系统	Windows 11 家庭版 24H2
处理器	AMD Ryzen 7 5800H
内存	16GB DDR4
显卡	NVIDIA GeForce RTX 3060 Laptop
网络环境	1000Mbps 有线/ 5Ghz Wi-Fi 无线
摄像头	720p 内置
并发客户端数	单一客户端

表 5.2 服务端测试环境配置

测试项	测试环境
操作系统	macOS Sequoia 15.4
处理器	Apple M1 Pro
内存	16GB 统一内存
显卡	Apple M1 Pro
网络环境	300 Mbps Wi-Fi 无线
并发客户端数	模拟 300 路客户端

5.2 功能测试

功能测试旨在验证系统客户端和服务端的各个模块是否按照设计需求正常工作。测试覆盖了用户界面交互、后台服务运行、数据采集与传输、指令接收与执

行等关键功能点。测试在 Windows 11 环境下进行。

5.2.1 客户端功能测试

客户端的功能测试主要关注其在考试机上的行为表现和数据采集传输的准确性。关键测试项包括应用生命周期管理、配置管理、媒体采集与推流、本地流媒体服务、API 服务与通信。

这里首先测试了应用的启动（单一实例锁定）、窗口隐藏、系统托盘图标显示及菜单功能。验证了通过托盘菜单退出时是否正确触发密码验证流程。其次测试了使用 `electron-store` 保存和读取配置信息的功能，并初步验证了配置文件的基础加密（混淆）是否生效，文件内容是否不易直接辨读。接着验证了 `FFmpeg` 子进程是否能成功启动，并正确捕获屏幕和摄像头画面。检查了画面合成（画中画）效果和本地 `Node-Media-Server (NMS)` 是否能接收到 `RTMP` 推流。初步验证了在支持硬件加速的机器上，`FFmpeg` 参数是否正确包含了硬件加速选项。最后验证了客户端能否成功建立与服务端的 `WebSocket` 连接，并进行身份认证。测试了客户端定期上报系统信息（CPU、内存、进程列表等，通过 `systeminformation` 库采集）是否正常，能否正确接收和响应服务器下发的指令，以及断线后自动重连机制是否有效。

5.2.2 服务端功能测试

服务端功能测试主要侧重于其核心业务逻辑和与多个客户端交互的能力。关键测试项包括客户端连接管理、Web 服务接口、消息处理与指令下发，测试在 `macOS` 环境通过 `ApiFox` 测试工具进行。

首先测试了服务端能否接收新的 `WebSocket` 连接，进行客户端身份验证（基于机器编号和密码/Token），并将在线客户端添加到管理列表。验证了客户端断开连接时，服务端能否及时检测并更新其在线状态。对于基本 Web 服务接口，测试了主要的 `RESTful API` 接口，如获取客户端列表、查询特定客户端信息等，验证接口响应是否正确，返回数据是否符合预期。也验证了服务端能否接收客户端上报的系统信息和流地址，并将其与对应的客户端关联。测试了通过管理界面（模拟或实际前端）向单个或全体客户端发送指令，验证指令能否被客户端正确接收和执行。测试结果如表 5.3 所示。

表 5.3 系统功能测试结果摘要

测试模块	主要测试功能	测试结果
客户端 - 应用管理	启动、单实例、窗口隐藏、托盘	通过
客户端 - 配置管理	配置存储、基础加密、加载	通过
客户端 - 媒体服务	FFmpeg 启动、采集、合成、推流至 NMS	通过
客户端 - 本地 NMS	NMS 运行、提供 HTTP-FLV 流	通过
客户端 - API 通信	WebSocket 连接、认证、信息上报、指令接收、重连	通过
服务端 - 连接管理	WebSocket 连接接收、认证、状态更新	通过
服务端 - Web 服务	RESTful API 调用 (如获取列表)	通过
服务端 - 消息/指令	接收客户端上报、指令下发	通过
服务端 - 流管理	接收/分发流地址	通过

5.3 性能测试

性能测试旨在评估系统在典型负载下的资源占用情况和响应能力，重点关注客户端对考试机性能的影响以及服务端的并发处理能力。

5.3.1 客户端性能评估

在 Windows 测试环境下，运行客户端进行屏幕和摄像头采集推流时，监测了其对考试机 CPU、内存和网络带宽的占用。测试表明，在开启硬件加速的情况下，客户端的 CPU 占用率能维持在较低水平（约 0.9% - 3.2%），内存占用稳定在 115.7 MB 左右。上行网络带宽主要取决于视频编码参数和屏幕活动情况，测试期间维持在 14 Mbps 范围内。这些数据表明，客户端在多数现代考试机上运行时，对正常考试操作的影响较小。长时间运行测试（运行了约 3 小时）未发现明显的性能衰退或内存泄漏。

5.3.2 服务端性能评估

服务端性能测试主要评估其能稳定支持的并发客户端连接数。通过模拟多个客户端连接并保持通信，观察服务端在不同连接数下的 CPU、内存和网络负载。初步测试显示，在表 5.2 所示的服务端配置下，系统能够稳定处理约 300 路并发客户端连接，此时服务端各项资源占用处于合理范围，指令下发和信息上报的延迟可控。当连接数进一步增加时，系统性能可能开始出现瓶颈，具体取决于服务器硬件和网络条件。

这里也测试了从 10 路客户端到 300 路客户端的平均接口响应时间，测试由 ApiFox 工具提供支持，测试时每个并发的进程循环 5 次，测试采用的参数如图 5.1 所示。测试结果如图 5.2 所示。



图 5.1 测试参数

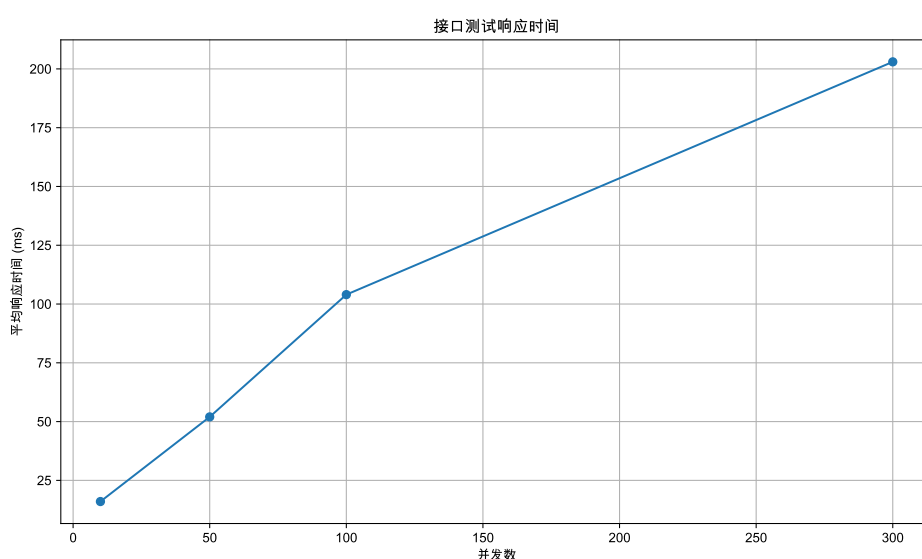


图 5.2 接口响应时间

5.4 基础安全检查

本阶段进行的基础安全检查主要关注系统实现的安全功能是否按设计工作，并未针对复杂的攻击手段进行渗透测试。首先在客户端验证了禁用系统快捷键（如 Alt+F4）和拦截窗口关闭事件是否生效，以及通过托盘菜单退出时密码验证是否正确。其次验证了尝试启动客户端的第二个实例时，新实例是否能被正确阻止并退出，也检查了存储的配置文件内容是否经过基础加密，不易直接读取。最后验证了客户端和管理员通过 WebSocket 或 API 连接时，使用错误的凭证是否会被服务器拒绝。上述测试均已通过，表明系统在设计层面的安全措施已初步实现并生效。

5.5 测试结论与展望

基于上述测试，本远程监考系统在 Windows 11 环境下实现了设计中的核心功能，包括客户端的屏幕/摄像头采集推流、系统信息上报、指令接收，以及服务端的客户端管理、消息通信和流地址分发。系统在典型负载下资源占用合理，能满足中小型规模的在线考试监控需求。基础安全检查也验证了防止非法退出和单实例运行等关键安全特性。

由于时间限制，本阶段主要测试在 Windows 环境进行。未来的工作应包括在其他关键操作系统（如 Ubuntu Linux）下进行更全面的功能、性能和兼容性测试，确保系统在不同平台上都能稳定可靠运行。此外，进一步的安全性评估，包括针对性的渗透测试和代码审计，也是提升系统鲁棒性的重要方向。在性能方面，可以进行更精细的调优和压力测试，以探索系统在高并发场景下的极限承载能力。

6 总结与展望

总的来看，本系统主要针对了上机考试背景下的各类需求，开发了一个相对基础的监考与推流系统，首先采用的 Electron 解决了目前大部分方案所不具备的高兼容性以及高跨平台能力，同时在安排了志愿者这一角色的情况下，提高了密码分发的安全性，也简化了参加上机考试的考生需要操作的流程。不过目前还存在一些不足和需要改进的地方，例如在反作弊方面还没做到自动化判定与非法操作的阻止，对于潜在的替考人员作弊暂时没有很好的解决方法。截至目前，本系统已具备了基本的功能，对监考系统的部署进行了大量的简化与自动化，方便了考时的实时监控与竞赛的直播功能。不过对于超大型上机考试的极高压场景并没有做到相应的测试，因此，在未来的工作中，还可以进一步完善系统的功能，提高系统的稳定性和安全性。

参考文献

- [1] AGGHEY A Z, MWINUKA L J, PANDHARE S M, et al. Detection of username enumeration attack on ssh protocol: Machine learning approach[J]. Symmetry, 2021, 13.
- [2] ZHANG C, QI D. How to parallelize rsync[J]. Journal of Physics: Conference Series, 2020, 1684.
- [3] FANG C, COTTRELL R L A, KISSEL E. When to use rsync[R]. Zettar Inc, Mountain View, CA (United States); SLAC National Accelerator Lab., Menlo Park, CA (United States); Lawrence Berkeley National Lab. (LBNL), Berkeley, CA (United States), 2021.
- [4] REDDY G P, NARAYANA A, KEERTHAN P K, et al. Multiple hashing using sha-256 and md5[C]//THAMPI S M, GELENBE E, ATIQUZZAMAN M, et al. Advances in Computing and Network Communications. Singapore: Springer Singapore, 2021.
- [5] 唐俊武, 南理勇, 左强. 在线考试系统开发中的几个问题及解决方法[J]. 计算机与数字工程, 2005(08): 144-147.
- [6] 邹宜金, 张美锋, 曹玲燕, 等. 基于 C/S 架构的电力设备数据库管理软件设计研究[J]. 电气技术与经济, 2023(04): 18-22.
- [7] 高厅权. 货运专线管控一体化系统的软件设计与实现[D]. 兰州交通大学, 2022.
- [8] KRAEV Y, FIRSOV G, KANDAKOV D. Authentication via rdp using electronic identifiers [C]//2021 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (ElConRus). 2021: 2361-2365.
- [9] BAI T, BIAN H, SALAHUDDIN M A, et al. Rdp-based lateral movement detection using machine learning[J]. Computer Communications, 2021, 165: 9-19.
- [10] LEE J, LEE H. Improving ssh detection model using ipa time and wgan-gp[J]. Computers and Security, 2022, 116.
- [11] WAHAB O F A, KHALAF A A M, HUSSEIN A I, et al. Hiding data using efficient combination of rsa cryptography, and compression steganography techniques[J]. IEEE Access, 2021, 9: 31805-31815.
- [12] LEE J, LEE H. An ssh predictive model using machine learning with web proxy session logs [J]. International Journal of Information Security, 2022, 21(2).
- [13] 周昌海, 樊玲, 谷云琦. 基于 C/S 架构 & 对称加密的图书馆有毒有害读物自动筛查及下架

系统软件设计[J]. 信息与电脑 (理论版), 2022, 34(24): 127-130.

- [14] 余晓青. 新时代意识形态网络舆情治理研究[D]. 福州: 福建师范大学, 2019.
- [15] 蔡克难. 多方面努力保证出版物质量[J]. 编辑学刊, 1994(4): 19-21.
- [16] 陈朝荣. 数据库的一种动态加密法[J]. 电脑编程技巧与维护, 1995(2): 57-58.
- [17] 刘艳. 基于 C/S 模式的远程网络视频监控系统的设计和实现[J]. 新型工业化, 2022, 12(09): 264-267.
- [18] 教文兵. 基于 H.264/AVC 屏幕录制回放系统[D]. 华中科技大学, 2012.
- [19] MARPE D, WIEGAND T, SULLIVAN G J. The h.264/mpeg4 advanced video coding standard and its applications[J]. IEEE Communications Magazine, 2006, 44(8): 134-143.
- [20] 裴世保, 李厚强, 俞能海. H.264/AVC 帧内预测模式选择算法研究[J]. 计算机应用, 2005, 25(8): 71-73.
- [21] 宋彬, 常义林, 李春林. H.264 帧间预测模式的快速选择算法[J]. 电子学报, 2007(4): 697-700.
- [22] HUANG Y W, HSIEH B Y, CHIEN S Y, et al. Analysis and complexity reduction of multiple reference frames motion estimation in h.264/avc[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2006, 16(4): 507-522.
- [23] SUH J, JEONG J. Fast sub-pixel motion estimation techniques having lower computational complexity[J]. IEEE Transactions on Consumer Electronics, 2004, 50(3): 968-973.
- [24] MALVAR H, HALLAPURO A, KARCZEWICZ M, et al. Low-complexity transform and quantization in h.264/avc[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2003, 13(7): 598-603.

致谢

这篇论文的顺利完成，离不开许多人的关心与帮助。在此，我谨向所有在我学习和生活中给予关怀、指导和支持的老师、同学、家人和朋友们致以最诚挚的感谢！

首先，衷心感谢我的导师。在毕业设计的选题、方案设计、系统开发、论文撰写等各个阶段，都给予了我悉心的指导和耐心的帮助。同时也感谢学院为我提供了良好的学习环境和丰富的学术资源。感谢各位任课老师在本科期间的辛勤教学和悉心培养，让我打下了坚实的专业基础。感谢我的家人对我一直以来的理解、支持和鼓励。最后也感谢所有在毕业设计过程中给予我帮助和支持的朋友们。

在本次毕业设计过程中，我不仅提升了专业技能，更锻炼了独立思考和解决问题的能力。虽然过程中遇到了许多困难和挑战，但也正是在这些经历中不断成长。

最后，再次向所有为本论文顺利完成付出过心血的老师、同学、家人和朋友们再次表示衷心的感谢！